# Rethinking Systematic Generalization: From Discriminative Benchmarks to Generative Evaluation


Chengwen Qi
National University of Singapore
Singapore, Singapore
qichengwen@u.nus.edu

Deheng Ye
Tencent
Shenzhen, China
ydyl1991@gmail.com

Yatao Bian*
National University of Singapore
Singapore, Singapore
ybian@nus.edu.sg


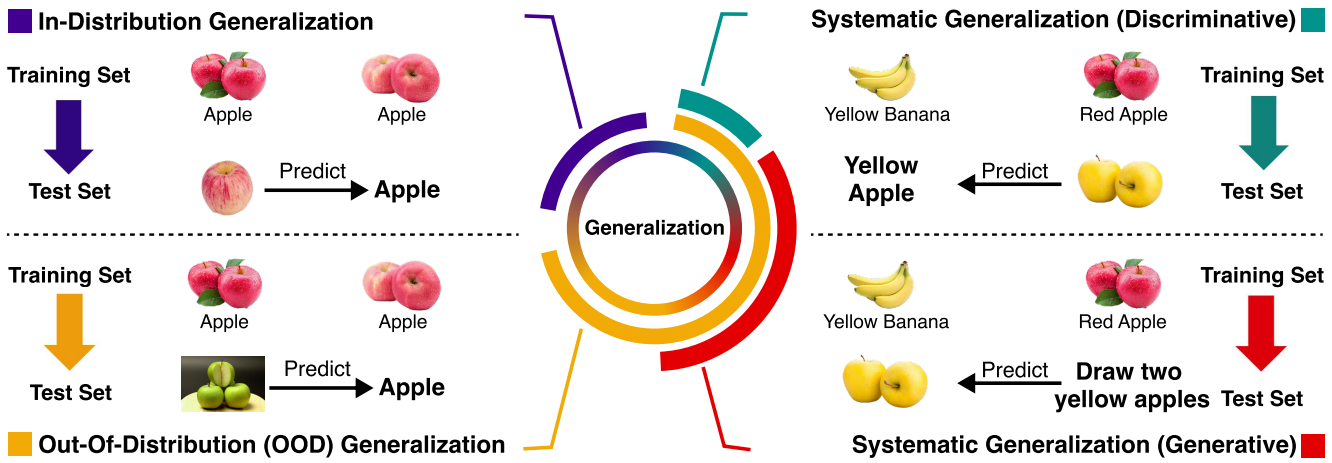


**Figure 1: Overview of different forms of generalization. Generalization can be broadly categorized into in-distribution and out-of-distribution (OOD) generalization. Systematic generalization represents a more challenging form of OOD generalization. Furthermore, generative systematic generalization is more demanding and encompasses a broader range of compositional scenarios than discriminative systematic generalization.**


## Abstract

Systematic generalization—the ability to understand and create novel combinations of known components—is widely regarded as a core aspect of human cognition. However, most existing benchmarks evaluate this capability using a discriminative paradigm, where models are asked to interpret or verify predefined combinations of atomic elements. Such evaluations cover only a tiny fraction of the compositional space and overlook the true challenge of systematic generalization. In this paper, we argue that systematic generalization should instead be evaluated from a generative perspective. Rather than validating predefined compositions, models should be required to autonomously construct solutions for high-level goals using atomic components. This formulation substantially expands the search space and better reflects real-world problem solving. We analyze key limitations of existing benchmarks and outline design principles for generative systematic generalization benchmarks, including controlled compositional structure, automatic verifiability, and goal-driven systematic synthesis. We further present a prototype experimental environment illustrating these principles. We believe generative systematic generalization evaluation provides a more faithful foundation for assessing compositional intelligence in modern AI systems.



## CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

## Keywords

Systematic Generalization, Open-Ended Evaluation, Large Models

**ACM Reference Format:**
Chengwen Qi, Deheng Ye, and Yatao Bian. 2026. Rethinking Systematic Generalization: From Discriminative Benchmarks to Generative Evaluation. In *Proceedings of* . ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX


## 1 Introduction

Systematic generalization [34], which evaluates the ability to systematically combine learned skills or tools in novel settings to address new problems, is a fundamental aspect of human intelligence [39]. As shown in Figure 1, systematic generalization is a form of out-of-distribution (OOD) generalization, but it has distinct characteristics. Specifically, it requires a model to systematically


*Corresponding author.



Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

,
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.


ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

compose previously acquired skills or knowledge to solve novel tasks. It evaluates a model's ability to understand a task and search within an extremely large combinatorial space to find the correct answer. Moreover, systematic generalization aligns closely with real-world tasks humans encounter every day, in which they must effectively apply previously learned skills to handle problems they have never faced before.

To evaluate this capability, previous research mainly focused on the text modality, started from the introduction of the SCAN dataset [33], which tested the ability of sequence-to-sequence models to compositionally translate simple natural language commands into action sequences. Subsequent works expanded this evaluation to more complex textual scenarios, including semantic parsing [28], situated language understanding [46], and knowledge graph reasoning [16, 27], consistently revealing that models struggle with productivity and novel combinations of seen elements. With the advent of Large Language Models (LLMs), researchers have further probed their compositional abilities across arithmetic [36] and mathematical reasoning [57], finding that performance significantly degrades on complex, multi-step composite tasks [54]. Beyond pure text, recent efforts have gradually introduced vision and agentic domains, assessing systematic generalization through multi-turn function calling [7], complex tool use [51], and text-to-image generation [19, 22, 44].

However, most of these benchmarks evaluate systematic generalization from a discriminative paradigm, where models are given pre-constructed combinations to parse or validate. This explicit combination of atomic elements may result in information leakage, leading to an overestimation of the model's performance. Furthermore, this setting diverges from real-world scenarios: humans are typically presented with a high-level goal rather than specific steps, requiring them to autonomously retrieve and combine existing skills to obtain the desired outcome. Therefore, in this paper, **we argue that systematic generalization should be evaluated from a generative perspective rather than a discriminative one**.

The contributions of this paper are summarized as follows: First, we identify three structural limitations of existing discriminative evaluation paradigms. Second, we show that generative evaluation can encompass a broader combinatorial space, enable the assessment of solution efficiency, and align better with real-world tasks. Third, we distill three essential design principles for future systematic generalization benchmarks and introduce a corresponding prototype experiment.

# 2 Why Systematic Generalization Matters and Why Now

## 2.1 Systematic Generalization as a Core Component of Human Intelligence

Human intelligence relies heavily on systematic compositionality, a core capacity that enables us to construct and comprehend an unbounded range of novel combinations from a limited set of basic components [57]. This remarkable capacity permeates nearly every facet of our daily lives.

In language, this ability allows us to understand and produce novel sentences, or even coin new words. It also extends to complex everyday tasks; for example, when using an unfamiliar software, humans can readily combine their prior knowledge of standard icons, menus, and interaction mechanisms—such as drag-and-drop—to accomplish tasks without explicit instructions.

In scientific and technological domains, systematic generalization drives innovation. By systematically recombining foundational components, researchers can extrapolate beyond known boundaries to uncover new physical laws and synthesize novel materials. For example, chemists create new molecules from basic atoms, and engineers create new tools from existing ones.

In contrast to these human capabilities, artificial neural networks have historically struggled with generative compositionality. As Fodor and Pylyshyn [14] argued, systematicity is the bedrock of human thought and early neural networks lacked this capacity. Despite recent advancements in AI, achieving systematic generalization remains a fundamental bottleneck today, as modern models still rely heavily on shortcut learning and pattern matching within their training distributions [5, 23, 29, 32, 41].

## 2.2 Beyond Standard Generalization: The Unique Challenge of Systematicity

In standard generalization, training and testing data are assumed to share the same distribution. OOD generalization relaxes this assumption to account for data shifts where the training and test distributions differ, yet their supports remain identical [2, 48, 52]. In this context, models adapt to variations in data frequency or distribution shift, while the fundamental boundaries of the data domain remain unchanged.

In contrast, systematic generalization presents a more challenging situation, requiring models to generalize to test distributions with different—potentially non-overlapping—support sets [52]. The defining feature of this paradigm is that the test data consists of entirely novel combinations of atomic components, each of which was observed individually during training. Unlike compositional generalization, which evaluates only the syntactic combination of atomic elements [45], systematic generalization assesses the ability to reason about novel combinations of skills—rarely or never observed together during training—and to systematically compose them to solve new problems. It requires models not only to interpret given combinations but also to select and synthesize atomic elements to propose effective solutions for the task. This generative and goal-oriented nature makes systematic generalization particularly challenging and more representative of real-world problem-solving, underscoring the need for evaluation paradigms that effectively assess this ability.

## 2.3 The Emerging Need for Generative Evaluation of Systematicity

The shift from discriminative to generative evaluation of systematic generalization is especially urgent at the present moment for two reasons: the rapid development of large models and the growing demand for AI in practical domains such as chemistry, biology and software engineering [1, 8, 12, 25, 37].

First, recent advances in generative models have significantly enhanced the capabilities of modern AI systems, making existing benchmarks insufficient for evaluating their performance. Early AI

models [13, 20, 30] were primarily designed for discriminative tasks and therefore often struggled with generative ones. Even in the early stages of LLMs [6, 42, 43, 49], AI models still struggled to follow user instructions, generate constrained outputs, and perform multi-step reasoning. Consequently, most existing systematic generalization benchmarks relied on discriminative evaluation protocols. However, after several years of rapid development, modern models have demonstrated remarkable capabilities in instruction following, tool use, and complex reasoning. As a result, traditional benchmarks are becoming increasingly inadequate for fully evaluating these systems' capabilities.

Second, the growing importance of AI in practical applications such as AI for Science underscores the need to evaluate systematic generalization generatively. In fields such as chemistry, biology, and materials science, AI systems are often tasked with predicting or generating novel materials or experimental designs [3, 4, 40, 53, 56]. These tasks require creating entirely new combinations of known elements rather than merely interpreting predefined ones, which highlights the importance of systematic generalization at the present stage.

Taken together, these developments underscore the urgent need to shift the evaluation of systematic generalization from discriminative to generative settings. This change offers a more appropriate and demanding benchmark for modern AI systems, while also aligning better with the requirements of real-world applications.

# 3 Limitations of Existing Systematic Generalization Benchmarks

## 3.1 The Saturation of Traditional Benchmarks

Although systematic generalization was first proposed in 1988 [14], it was not until 2018 that researchers introduced **SCAN** (**S**implified version of the **C**omm**A**I **N**avigation tasks), a benchmark specifically designed to evaluate the systematic generalization capabilities of neural networks [33].

In SCAN dataset, models are trained to map atomic commands to action sequences—for example, converting `jump` → `JUMP`, `run left` → `LTURN RUN`, and `walk twice` → `WALK WALK`. Subsequently, models are evaluated on novel or longer combinations of atomic elements that were not encountered during training. The authors evaluated LSTMs on the SCAN dataset and found that while they perform well on standard generalization tasks (e.g., random splits of training and test data) given sufficient training examples, they struggle with systematic generalization tasks, such as longer sequences. Following SCAN, many datasets have been proposed to evaluate the systematic generalization capabilities of AI systems, including gSCAN for situated language understanding [46], COGS for semantic parsing [28], CFQ [27] and MCWQ [11] for knowledge base question answering, and MATHTRAP for mathematical reasoning [57].

Although these tasks were challenging when first introduced, they are no longer difficult for modern AI models, especially reasoning oriented models such as OpenAI o1 [24], Gemini [10], DeepSeek-R1 [17], and Qwen [55]. Recent studies have shown that these models can effectively handle semantic parsing, arithmetic reasoning, and command-to-action sequence generation, even for extremely long inputs [21, 31, 35].

## 3.2 Irreducible Noise Factors in Multimodal Benchmarks

To address the limitations of early benchmarks, recent work has introduced more realistic environments for evaluating systematic generalization. These benchmarks often feature richer settings and multimodal observations, such as web automation and game play. Examples include CompWoB [15], which evaluates compositional web tasks built from basic operations (e.g., entering information and submitting forms); UI-NEXUS [18], which evaluates mobile agents that execute instructions by combining atomic skills; and the game *Baba Is You* [9, 50], which tests reasoning in environments where the rules can be changed dynamically.

Although such benchmarks are more representative of real-world scenarios, they also introduce substantial intervening factors that complicate the evaluation of systematic generalization. In multimodal settings, model performance may depend heavily on ancillary capabilities such as visual perception, entity recognition, or object manipulation. Consequently, poor performance on these tasks may reflect limitations in those supporting abilities rather than a genuine failure of systematic generalization. Ultimately, while valuable for assessing systematic generalization in practical settings, multimodal environments struggle to isolate it as a core cognitive ability due to irreducible intervening factors.

## 3.3 The Inherent Flaws of Discriminative Evaluation Paradigm

Beyond model saturation and environmental complexity, the most significant weakness of existing evaluation protocols is their reliance on a discriminative paradigm. We highlight three limitations:

First, limited coverage of the compositional space. Discriminative evaluation requires researchers to explicitly enumerate test combinations of atomic elements. As a result, models are assessed on only a tiny fraction of the underlying compositional space (see Sec 4.1 for further details). This not only limits benchmark coverage but also lowers task difficulty, since the relevant combinations are already provided to the model, and the model only needs to parse, classify, or verify them. The core challenge of systematic generalization—actively retrieving and composing atomic skills to solve a given task—is therefore bypassed.

Second, an inability to evaluate solution quality and efficiency. In real-world scenarios, a single problem often has multiple valid solutions with substantially different operational costs. For example, oxygen can be produced either through water electrolysis or through cryogenic air separation. Although both methods achieve the same goal, electrolysis is far more expensive, making cryogenic air separation the preferred industrial approach. A discriminative benchmark can verify whether a predefined combination is valid, but it cannot compare the efficiency of different solutions.

Finally, misalignment with real-world problems. Humans are rarely given predefined action sequences and asked merely to judge whether they are correct. Instead, real-world problems are typically open-ended and goal-driven, requiring agents to independently identify, retrieve, and integrate the relevant knowledge and skills. Consequently, discriminative benchmarks fall short of adequately reflecting the kind of compositional intelligence needed in practice.

# 4 The Generative Paradigm for Evaluating Systematic Generalization

## 4.1 Scaling the Combinatorial Search Space

A fundamental advantage of assessing systematic generalization from a generative perspective is its ability to cover a much broader and inherently more realistic range of compositional scenarios. Unlike discriminative approaches, which passively evaluate models on a limited set of predefined combinations, the generative approach requires the model to actively explore an exponentially expanding space to get the correct solution.

To understand this discrepancy theoretically, consider a task that requires combining atomic elements from a defined set $A = \{a_1, a_2, \ldots, a_N\}$ of size $N$. In a traditional discriminative evaluation, researchers manually construct a test set $T$ consisting of $M$ specific combinations, typically with a fixed length or bounded complexity $L$. As a result, the model is evaluated on only $M$ points in the compositional space. As $M$ needs to remain computationally and practically manageable, it constitutes a extremely small fraction of all possible combinations ($M \ll N$). This sparsity allows models to achieve high performance by relying on surface-level heuristics or spurious correlations, rather than mastering a genuine understanding of the underlying mechanisms.

In stark contrast, the generative paradigm does not provide the model with a predefined sequence to verify or translate. Instead, it provides a high-level goal $G$, and requires the model to autonomously select and combine atomic elements to construct a valid solution $S = (a_{i_1}, a_{i_2}, \ldots, a_{i_L})$. To achieve this, the model need to implicitly or explicitly navigate a vast combinatorial search space $\mathcal{H}$. If the solution requires up to $L_{max}$ steps, the size of this search space grows exponentially:

$$|\mathcal{H}| = \sum_{l=1}^{L_{max}} N^l = \frac{N(N^{L_{max}} - 1)}{N - 1} \approx O(N^{L_{max}}).$$

To illustrate the scale of this combinatorial explosion, consider a relatively constrained environment with just 15 atomic elements ($N = 15$). If the task requires the model to generate a solution sequence of up to 10 steps ($L_{max} = 10$), it will need to navigate a search space of more than 600 billion possible combinations, not to mention that real-world tasks often involve solutions containing far more than 10 steps.

## 4.2 Beyond correctness: evaluating solution efficiency

Beyond simply searching the combination space to find a valid solution, the generative paradigm introduces an important additional criterion for evaluation: solution efficiency.

In discriminative tasks, evaluation is largely binary, focusing only on whether a predefined combination is correct. However, in many real-world scenarios, multiple plans may satisfy a given high-level goal $G$, while differing substantially in execution overhead. By requiring the model to explicitly construct the full solution path $S$, the generative framework enables us to assess not only the correctness of the final outcome but also the efficiency of the solution itself.

To formalize this efficiency evaluation, we can assign a predefined cost to each atomic action. For instance, let each atomic element $a_n \in A$ be associated with a specific scalar cost $c_n > 0$. For any valid solution sequence $S = (a_{i_1}, a_{i_2}, \ldots, a_{i_L})$, the overall execution cost can be calculated by summing the costs of each step. The total cost $C(S)$ is defined as the linear sum:

$$C(S) = \sum_{j=1}^{L} c_{i_j}.$$

This metric offers a straightforward way to compare different solutions by penalizing redundant, or expensive steps. Consequently, the generative paradigm evaluates not only whether a model has the compositional understanding required to reach a valid state, but also whether it can systematically derive the most efficient strategy.

## 4.3 Aligning with Real-World Problems

The generative paradigm aligns naturally with real-world problems, which are usually goal-oriented and open-ended. Instead of following predefined steps, models need to generate solutions through systematic composition of atomic components. This characteristic appears across many domains. For instance, a marketer designs advertising materials for a specific group of audience, or a software engineer combines existing tools and skills to implement a new feature. It is particularly evident in scientific research: when designing materials with specific properties, researchers do not simply select or verify existing molecular combinations. Instead, they construct new structures from fundamental building blocks and evaluate whether the resulting configurations satisfy the desired requirements.

Additionally, the shift toward generative evaluation is increasingly driven by practical demands. There is growing interest in deploying AI systems as autonomous agents in real-world environments. These systems are expected to address open-ended problems in a manner similar to humans. However, evaluating them with discriminative benchmarks creates a mismatch between reported capabilities and their practical usefulness. Consequently, shifting toward generative evaluations is becoming increasingly important for assessing and supporting the reliable deployment of autonomous agents.

## 4.4 Essential Properties of Generative Systematic Generalization benchmarks

To accurately assess systematic generalization on modern AI systems and ensure that evaluation distinguishes genuine compositional reasoning from shallow pattern matching, a generative benchmark should satisfy several essential properties. We outline these key characteristics below and present a prototype experiment designed around these principles in Appendix A.

**Controlled compositional structure.** A meaningful evaluation of systematic generalization requires a clearly defined set of atomic components and compositional rules. The training and evaluation splits should be constructed such that individual primitives are observed during training, while their higher-order compositions remain novel at test time. This controlled structure allows

researchers to isolate the model's ability to recombine known elements into novel configurations, rather than measuring gaps in knowledge of the primitives themselves.

**Verifiability.** Generative benchmarks should support reliable and automatic verification of generated solutions. Unlike discriminative tasks, where predictions can be directly compared with predefined labels, generative tasks may admit multiple valid solutions. Therefore, the evaluation environment should provide a mechanism to automatically determine whether a generated solution successfully achieves the goal. Ideally, this verification process should minimize confounding factors unrelated to systematic reasoning, such as complex perception or low-level control.

**Goal-driven systematic synthesis.** Finally, the benchmark should require models to synthesize solutions in a goal-directed manner. Beyond merely combining skills that were explicitly demonstrated during training, models should be able to identify and exploit implicit properties of the available atomic components. In other words, a model should recognize that certain primitives possess capabilities that were not explicitly highlighted in training, and selectively apply them when they become relevant to a task. This form of goal-driven synthesis reflects a deeper level of systematic generalization, where models actively reason about how existing skills can be repurposed and recombined to satisfy new goals.

## 5 Vision of Successes

Beyond improving benchmark design, the generative evaluation paradigm also highlights a broader shift in how we envision future AI systems. Rather than functioning solely as predictive models trained to reproduce patterns in existing data, AI systems may increasingly act as generative problem solvers that actively construct new solutions through systematic composition. This perspective suggests several promising directions in which generative systematic generalization could transform practical applications.

### 5.1 From Predictive Models to Autonomous Agents

Scientific research is fundamentally an exercise in generative systematic generalization. The pursuit of new knowledge rarely involves selecting from a pre-defined set of actions; rather, it requires researchers to systematically synthesize established physical laws, existing experimental data, and foundational theories to formulate and validate novel hypotheses. In this context, scientific discovery relies on taking atomic components of knowledge and combining them in novel ways to solve previously unencountered problems.

If AI systems can master this generative capacity, they will become powerful tools for autonomous scientific discovery. By applying compositional reasoning to existing knowledge, this approach shifts the paradigm from human-constrained, trial-and-error screening to the rapid, systematic evaluation of complex structures, significantly accelerating scientific discovery. Pioneering breakthroughs like AlphaFold [26] for protein structure prediction, GNoME [38] for inorganic crystal discovery, and AlphaGo [47] for go strategy generation already demonstrate AI's profound ability to navigate massive search spaces, accomplishing in days what would traditionally take human experts years. Building on these successes, generative systematic generalization enables models to extrapolate beyond known data distributions. By composing physical and chemical principles, these systems can autonomously design novel configurations, facilitating the rational exploration of uncharted chemical spaces and accelerating the discovery of new materials.

### 5.2 Autonomous Generation of Novel Technical Solutions

Equipped with generative systematic generalization, AI systems could autonomously discover novel technical solutions that have been overlooked by humans before by recombining existing knowledge and foundational elements. A practical example of this is the smartphone, which integrated distinct functions like communication, photography, and audio playback into one platform. If AI can replicate this process, it could systematically identify novel combinations of existing technologies. Exploring these technological design spaces would not only accelerate the discovery of novel tools, but also lead to massive economic impact.

### 5.3 Systematic Workflow Optimization

Beyond evaluating accuracy, the generative evaluation paradigm enables the assessment of operational efficiency. By assigning specific time or resource costs to each atomic step, we can evaluate a model's capacity to not merely find a valid solution, but to discover the optimal pathway to achieve it. Instead of solely focusing on task completion, the model is optimized to reach the goal using the minimum necessary steps.

If AI systems achieve this level of systematic generalization, they can be deployed to optimize existing human workflows. In many domains, established operations often contain redundant steps or structural inefficiencies that are easily overlooked. An AI capable of systematic generalization could automatically analyze these workflows and reorganize them into an optimal sequence. Ultimately, this would significantly enhance productivity and resource allocation across various industries, yielding substantial social and economic benefits.

## References


[1] Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J Ballard, Joshua Bambrick, et al. 2024. Accurate structure prediction of biomolecular interactions with AlphaFold 3. *Nature* 630, 8016 (2024), 493–500.

[2] Shai Ben-David and Ruth Urner. 2014. Domain adaptation–can quantity compensate for quality? *Annals of Mathematics and Artificial Intelligence* 70, 3 (2014), 185–202.

[3] Yatao Bian, Huaijin Wu, and Junchi Yan. 2026. Deep learning for affinity prediction and interface prediction in molecular interactions. *Deep Learning in Drug Design* (2026), 283–296.

[4] Yatao Bian, Nianzu Yang, Jiaxiang Wu, and Junchi Yan. 2026. Deep learning for complex structure prediction in molecular interactions. In *Deep Learning in Drug Design*. Elsevier, 297–308.

[5] Stella Biderman, Usvsn Prashanth, Lintang Sutawika, Hailey Schoelkopf, Quentin Anthony, Shivanshu Purohit, and Edward Raff. 2023. Emergent and predictable memorization in large language models. *Advances in Neural Information Processing Systems* 36 (2023), 28072–28090.

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[7] Mingyang Chen, Tianpeng Li, Fan Yang, Hao Liang, Bin CUI, Wentao Zhang, Zenan Zhou, et al. 2025. Facilitating Multi-turn Function Calling for LLMs via Compositional Instruction Tuning. In *The Thirteenth International Conference on Learning Representations*.

[8] Mouyang Cheng, Chu-Liang Fu, Ryotaro Okabe, Abhijatmedhi Chotrattanapituk, Artittaya Boonkird, Nguyen Tuan Hung, and Mingda Li. 2026. Artificial intelligence-driven approaches for materials design and discovery. *Nature Materials* (2026), 1–17.

[9] Nathan Cloos, Meagan Jens, Michelangelo Naim, Yen-Ling Kuo, Ignacio Cases, Andrei Barbu, and Christopher J Cueva. 2024. Baba Is AI: Break the Rules to Beat the Benchmark. In *ICML 2024 Workshop on LLMs and Cognition*. https://openreview.net/forum?id=jjN1A9CZn4

[10] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs.CL] https://arxiv.org/abs/2507.06261

[11] Ruixiang Cui, Rahul Aralikatte, Heather Lent, and Daniel Hershcovich. 2022. Compositional generalization in multilingual semantic parsing over Wikidata. *Transactions of the Association for Computational Linguistics* 10 (2022), 937–955.

[12] Giannis Daras, Jeffrey Ouyang-Zhang, Krithika Ravishankar, Constantinos Costis Daskalakis, Adam Klivans, and Daniel Jesus Diaz. 2025. Ambient Proteins - Training Diffusion Models on Noisy Structures. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=Xll01vw606

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[14] Jerry A Fodor and Zenon W Pylyshyn. 1988. Connectionism and cognitive architecture: A critical analysis. *Cognition* 28, 1-2 (1988), 3–71.

[15] Hiroki Furuta, Yutaka Matsuo, Aleksandra Faust, and Izzeddin Gur. 2024. Exposing Limitations of Language Model Agents in Sequential-Task Compositions on the Web. *Transactions on Machine Learning Research* (2024). https://openreview.net/forum?id=Y9kAsYIjYc

[16] Shengxiang Gao, Jey Han Lau, and Jianzhong Qi. 2025. Beyond seen data: Improving kbqa generalization through schema-guided logical form generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 8764–8783.

[17] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.

[18] Yuan Guo, Tingjia Miao, Zheng Wu, Pengzhou Cheng, Ming Zhou, and Zhuosheng Zhang. 2025. Atomic-to-Compositional Generalization for Mobile Agents with A New Benchmark and Scheduling System. *arXiv preprint arXiv:2506.08972* (2025).

[19] Xu Han, Linghao Jin, Xiaofeng Liu, and Paul Pu Liang. 2025. Progressive Compositionality in Text-to-Image Generative Models. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=S85PP4xjFD

[20] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[21] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[22] Kaiyi Huang, Kaiyue Sun, Enze Xie, Zhenguo Li, and Xihui Liu. 2023. T2i-compbench: A comprehensive benchmark for open-world compositional text-to-image generation. *Advances in Neural Information Processing Systems* 36 (2023), 78723–78747.

[23] Nurit Cohen Inger, Yehonatan Elisha, Bracha Shapira, Lior Rokach, and Seffi Cohen. 2025. Forget What You Know about LLMs Evaluations - LLMs are Like a Chameleon. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 21664–21677. doi:10.18653/v1/2025.emnlp-main.1098

[24] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] https://arxiv.org/abs/2412.16720

[25] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[26] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. 2021. Highly accurate protein structure prediction with AlphaFold. *nature* 596, 7873 (2021), 583–589.

[27] Daniel Keysers, Nathanael Scharli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, et al. 2020. Measuring Compositional Generalization: A Comprehensive Method On Realistic Data. In *8th International Conference on Learning Representations, ICLR 2020*.

[28] Najoung Kim and Tal Linzen. 2020. COGS: A Compositional Generalization Challenge Based on Semantic Interpretation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9087–9105.

[29] Muhammed Yusuf Kocyigit, Eleftheria Briakou, Daniel Deutsch, Jiaming Luo, Colin Cherry, and Markus Freitag. 2025. Overestimation in LLM Evaluation: A Controlled Large-Scale Study on Data Contamination's Impact on Machine Translation. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=MpjtvkvXDo

[30] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* 25 (2012).

[31] Jiayi Kuang, Haojing Huang, Yinghui Li, Xinnian Liang, Zhikun Xu, Yangning Li, Xiaoyu Tan, Chao Qu, Meishan Zhang, Ying Shen, and Philip S. Yu. 2025. Atomic Thinking of LLMs: Decoupling and Exploring Mathematical Reasoning Abilities. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=iBFfb6bGOz

[32] Lukas Kuhn, Sari Sadiya, Jörg Schlötterer, Florian Buettner, Christin Seifert, and Gemma Roig. 2025. Efficient unsupervised shortcut learning detection and mitigation in transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2217–2226.

[33] Brenden Lake and Marco Baroni. 2018. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International conference on machine learning*. PMLR, 2873–2882.

[34] Brenden M Lake and Marco Baroni. 2023. Human-like systematic generalization through a meta-learning neural network. *Nature* 623, 7985 (2023), 115–121.

[35] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2023), 42330–42357.

[36] Qing Li, Siyuan Huang, Yining Hong, Yixin Zhu, Ying Nian Wu, and Song-Chun Zhu. 2023. A Minimalist Dataset for Systematic Generalization of Perception, Syntax, and Semantics. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=kIPyTuEZuAK

[37] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature machine intelligence* 6, 5 (2024), 525–535.

[38] Amil Merchant, Simon Batzner, Samuel S Schoenholz, Muratahan Aykol, Gowoon Cheon, and Ekin Dogus Cubuk. 2023. Scaling deep learning for materials discovery. *Nature* 624, 7990 (2023), 80–85.

[39] Philipp Mondorf, Shijia Zhou, Monica Riedler, and Barbara Plank. 2026. Compositional-ARC: Assessing Systematic Generalization in Abstract Spatial Reasoning. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=h497VpgFKd

[40] Yifan Niu, Ziqi Gao, Tingyang Xu, Yang Liu, Yatao Bian, Yu Rong, Junzhou Huang, and Jia Li. 2025. InversionGNN: A Dual Path Network for Multi-Property Molecular Optimization. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=nYPuSzGE3X

[41] Chengwen Qi, Bowen Li, Binyuan Hui, Bailin Wang, Jinyang Li, Jinwang Wu, and Yuanjun Laili. 2023. An Investigation of LLMs' Inefficacy in Understanding Converse Relations. In *The 2023 Conference on Empirical Methods in Natural Language Processing*. https://openreview.net/forum?id=aN8zkE15Nx

[42] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).

[43] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[44] Arijit Ray, Filip Radenovic, Abhimanyu Dubey, Bryan Plummer, Ranjay Krishna, and Kate Saenko. 2023. Cola: A benchmark for compositional text-to-image retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 46433–46445.

[45] Patrik Reizinger, Szilvia Ujváry, Anna Mészáros, Anna Kerekes, Wieland Brendel, and Ferenc Huszár. 2024. Position: Understanding LLMs Requires More Than Statistical Generalization. In *International Conference on Machine Learning*. PMLR, 42365–42390.

[46] Laura Ruis, Jacob Andreas, Marco Baroni, Diane Bouchacourt, and Brenden M Lake. 2020. A benchmark for systematic generalization in grounded language understanding. *Advances in neural information processing systems* 33 (2020), 19861–19872.

[47] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature* 529, 7587 (2016), 484–489.

[48] Masashi Sugiyama, Matthias Krauledat, and Klaus-Robert Müller. 2007. Covariate shift adaptation by importance weighted cross validation. *Journal of Machine Learning Research* 8, 5 (2007).
[49] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
[50] Fien van Wetten, Aske Plaat, and Max van Duijn. 2025. Baba is llm: reasoning in a game with dynamic rules. In *International Joint Conference on Computational Intelligence*. Springer, 60–77.
[51] Jingwei Wang, Zai Zhang, Hao Qian, Chunjing Gan, Binbin Hu, Ziqi Liu, Zhiqiang Zhang, Jun Zhou, Bin Shi, and Bo Dong. 2025. Enhancing LLM Tool Use with High-quality Instruction Data from Knowledge Graph. *arXiv preprint arXiv:2506.21071* (2025).
[52] Thaddäus Wiedemer, Prasanna Mayilvahanan, Matthias Bethge, and Wieland Brendel. 2023. Compositional Generalization from First Principles. In *Thirty-seventh Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=LqOQ1uJmSx
[53] Guikun Xu, Yankai Yu, Yongquan Jiang, Yan Yang, and Yatao Bian. 2025. CoFM: Molecular Conformation Generation via Flow Matching in SE(3)-Invariant Latent Space. In *ICML 2025 Generative AI and Biology (GenBio) Workshop*. https://openreview.net/forum?id=C0jrjy4F1D
[54] Zhuoyan Xu, Zhenmei Shi, and Yingyu Liang. 2024. Do Large Language Models Have Compositional Ability? An Investigation into Limitations and Scalability. In *First Conference on Language Modeling*. https://openreview.net/forum?id=iI1CzEhEMU
[55] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388
[56] Ziqiao Zhang, Yatao Bian, Ailin Xie, Pengju Han, and Shuigeng Zhou. 2023. Can pretrained models really learn better molecular representations for ai-aided drug discovery? *Journal of Chemical Information and Modeling* 64, 7 (2023), 2921–2930.
[57] Jun Zhao, Jingqi Tong, Yurong Mou, Ming Zhang, Qi Zhang, and Xuan-Jing Huang. 2024. Exploring the Compositional Deficiency of Large Language Models in Mathematical Reasoning Through Trap Problems. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 16361–16376.

# A The Design of Prototype Experiments

## A.1 Basic Experimental Design

The prototype experiment is formulated as a discrete state transformation game. The agent is asked to synthesize an object with target state by applying a sequence of actions to an initial set of objects. The environment is fully deterministic, requiring multi-step planning, systematic reasoning, and strict adherence to action rules. Detailed experimental settings are provided in Table 1.

## A.2 Evaluation of Systematicity via Resource Scarcity

To evaluate the models' capacity for systematic generalization, we design a resource scarcity experiment, which targets one specific facet of this broader capability (i.e., discovering latent pathways not explicitly provided in the rules). The agent must synthesize a target object requiring a specific number of basic components, but the environment deliberately provides an insufficient quantity. To succeed, the model must discover and exploit an asymmetry between the SYNTHESIS and DECOMPOSITION tables: synthesizing a particular intermediate state consumes $n$ basic objects, while decomposing it yields $m > n$. By repeating this cycle, the agent can replenish its resources and eventually reach the target. An example is shown in Figure 2.

This design provides a straightforward test of systematicity. Because resource generation is not explicitly specified in the rules (Table 1), a successful agent should be able to infer how to combine basic actions to achieve this subgoal. Failure to do so suggests that the model lacks systematic generalization capabilities: it can apply basic tools individually, but cannot combine them systematically.

## A.3 Preliminary Experimental Results

We evaluate six state-of-the-art LLMs, including both proprietary and open-source models, on both the generative and discriminative tasks using a set of 100 instances [1]. The results are summarized in Table 2.

As shown, there is a significant performance gap between the two evaluation paradigms. On the same data, switching from discriminative to generative evaluation causes accuracy to drop by 2–5× (e.g., Gemini 3.1 Pro: 92% → 52%, GPT 5.4: 78% → 23%, DeepSeek V3.2: 53% → 6%), while average output length increases by 2–4× (e.g., GPT 5.4: 5,521 → 23,117 tokens, Grok 4.20: 11,497 → 28,078 tokens). Interestingly, despite the discriminative task involving longer action sequences on average, models still achieve substantially higher accuracy. This further confirms that generative evaluation is more challenging, as it requires models to actively explore a vast compositional space rather than merely verifying predefined combinations. The code is available in our GitHub repository[2].

[1] Note that GPT-5.4 Pro is not included in our evaluation due to budget constraints.
[2] https://github.com/BlueWhaleLab/GenSG

```
Goal: PYX
Objects:
{"name": "O1", "initial_state": "C", "locked": false},
{"name": "O2", "initial_state": "E", "locked": false},
{"name": "O3", "initial_state": "C", "locked": false},
{"name": "O4", "initial_state": "A", "locked": false},
{"name": "O5", "initial_state": "G", "locked": false},
{"name": "O6", "initial_state": "B", "locked": false}

synthesis table:
"I": ["DAAF", "DAC"],
"K": ["CCAF"],
"L": ["FCC"],
"M": ["EEEG", "EG"],
"N": ["ECDB", "EGD"],
"P": ["BGE", "DB"],
"Q": ["FG"],
"R": ["AECD", "EDCA"],
"S": ["EID"],
"T": ["BG", "LRL"],
"U": ["GIAA"],
"V": ["CFBP"],
"W": ["KNAB"],
"X": ["FV", "QWRL"],
"Y": ["LUCQ"],
"Z": ["MGN"]

decomposition table:
"I": ["G", "G", "E"],
"K": ["F", "F", "B", "B"],
"L": ["A", "D", "G"],
"M": ["A", "C", "E", "D"],
"N": ["D", "F"],
"P": ["G", "E"],
"Q": ["B", "B"],
"R": ["B", "B", "B", "G"],
"S": ["Q", "A", "G"],
"T": ["B", "Q"],
"U": ["M", "R", "C", "G"],
"V": ["E", "Q", "E"],
"W": ["D", "B", "P", "N"],
"X": ["C", "B"],
"Y": ["B", "W", "S"],
"Z": ["V", "U"]

One feasible solution:
"SYNTHESIS(O2, O5)", (H1: M)
"DECOMPOSITION(H1)", (J1: A, J2: C, J3: E, J4: D)
"REVERT(J1)", (J1: G)
"SYNTHESIS(J3, J1)", (H2: M)
"DECOMPOSITION(H2)", (J5: A, J6: C, J7: E, J8: D)
"REVERT(J5)", (J5: G)
"SYNTHESIS(J7, J5)", (H3: M)
"DECOMPOSITION(H3)", (J9: A, J10: C, J11: E, J12: D)
"REVERT(J9)", (J9: G)
"SYNTHESIS(J11, J9)", (H4: M)
"DECOMPOSITION(H4)", (J13: A, J14: C, J15: E, J16: D)
"REVERT(O4)", (O4: G)
"FORWARD(O1)", (O1: E)
"SYNC(J8, O4)", (J8: G)
"REVERT(J13)", (J13: G)
"REVERT(O3)", (O3: F)
"SYNC(J16, O4)", (J16: G)
"REVERT(J14)", (J14: F)
"SYNTHESIS(J15, O4)", (H5: M)
"SYNTHESIS(O1, J8, J12)", (H6: N)
"SYNTHESIS(H5, J13, H6)", (H7: Z)
"DECOMPOSITION(H7)", (J17: V, J18: U)
"SYNTHESIS(O6, J16)", (H8: T)
"DECOMPOSITION(H8)", (J19: B, J20: Q)
"SYNTHESIS(J4, J19)", (H9: P)
"SYNTHESIS(O3, J2, J6)", (H10: L)
"SYNTHESIS(H10, J18, J10, J20)", (H11: Y)
"SYNTHESIS(J14, J17)", (H12: X)
"SYNTHESIS(H9, H11, H12)", (H13: PYX)
```

**Figure 2: An example illustrating the asymmetry between SYNTHESIS and DECOMPOSITION. By recognizing that an intermediate state (M) requires only two basic objects to synthesize but yields four when decomposed, the agent can repeatedly apply the synthesis–decomposition cycle to generate additional resources and ultimately obtain the target.**

**Table 1: The Rules of the Proposed Task**

| Component / Action | Rules and Descriptions |
|---|---|
| | **Objects and States** |
| **Basic States (STATES0)** | A, B, C, D, E, F, G. |
| **Higher-level States** | I, K, L, M, N, P, Q, R, S, T, U, V, W, X, Y, Z. |
| **Naming Conventions** | (1) The initial objects are denoted by the prefix O (e.g., O1, O2).<br>(2) Synthesized objects start with H (e.g., H1, H2).<br>(3) Decomposed objects start with J (e.g., J1, J2).<br>(4) Once an object is consumed (via Synthesis or Decomposition), its name and ID are permanently retired. New objects always receive strictly increasing IDs. |
| **Lock Status** | Objects can be [LOCKED] or unlocked. Locked objects cannot be directly modified by basic actions or as the direct target of a SYNC action. |
| **Basic Actions:** These actions can **only** be applied to objects currently in STATES0 (A–G). Objects originating from decomposition can be subjected to these actions as long as their new states belong to STATES0. | |
| **FORWARD(obj)** | Advances the state 2 steps cyclically. (A→C, B→D, C→E, D→F, E→G, F→A, G→B). |
| **BACKWARD(obj)** | Reverses the state 2 steps cyclically. (C→A, D→B, etc.) |
| **REVERT(obj)** | Applies a specific mapping: A→G, B→E, C→F, D→D, E→B, F→C, G→A. |
| **RESET(obj)** | Changes the state of the object back to A. |
| **Advanced Actions:** These actions connect two objects. There is no restriction on the states of the objects involved, except that the target cannot be locked. | |
| **SYNC(target, source)** | Copies the state of the source to the target.<br>(1) The target cannot be locked.<br>(2) If the target has a bound target (via BIND), the updated state propagates to that bound object, **ignoring the bound target's lock status**. |
| **BIND(target, source)** | Binds the source to the target.<br>(1) **Propagation**: Any state-changing basic action applied to the source will automatically propagate to the target.<br>(2) **Unchangeable**: Once bound, the relationship is permanent.<br>(3) **Exclusivity**: Each object can participate in a maximum of **one** binding relationship (it can be a source OR a target, but not both).<br>(4) **Removal**: An object can be freed from a binding relationship and reused in a new relationship **only** if its current binding partner is destroyed via a SYNTHESIS or DECOMPOSITION action.<br>(5) **Applicability**: Binding relationships can be established between objects in any state. However, if the target is not in STATES0, applying a basic action to the source will result in an error, as the action cannot successfully propagate to a non-basic state target. |
| | **Synthesis and Decomposition Actions** |
| **SYNTHESIS(obj1, ...)** | Combines multiple objects together.<br>(1) Any objects can be synthesized together without restriction. The provisional result is determined by concatenating the states of the specified objects in the exact order they are provided.<br>(2) If the concatenated string exists in the **Synthesis Table**, it successfully resolves to the new higher-level state. (Note: A single higher-level state can have multiple valid synthesis paths defined in the table).<br>(3) Source objects are permanently destroyed, severing any of their existing binding relationships.<br>(4) The newly created H-prefix object is locked by default. |
| **DECOMPOSITION(obj)** | Breaks apart an object into multiple smaller parts based on the **Decomposition Table**.<br>(1) Unlike Synthesis, each element in the Decomposition Table has exactly **one** predefined decomposition result.<br>(2) You can **only** apply this action to objects whose states exist as keys in the Decomposition Table. Attempting to decompose any other object will cause an error.<br>(3) The source object is permanently destroyed.<br>(4) Newly generated J-prefix objects are unlocked if their resulting state is in STATES0, otherwise they are locked. |

**Table 2: Performance of LLMs on generative and discriminative tasks. Accuracy is reported as a percentage (%). Avg. Output Length is measured in tokens. †Results unavailable due to exceeding the model's maximum output length.**

| Model | Discriminative Acc. | Discriminative Avg. Output Len. | Generative Acc. | Generative Avg. Output Len. |
|---|---|---|---|---|
| GPT 5.4 | 78.00 | 5,520.57 | 23.00 | 23,116.56 |
| Gemini 3.1 Pro | 92.00 | 10,996.72 | 52.00 | 20,248.39 |
| Grok 4.20 | 84.00 | 11,497.30 | 13.00 | 28,078.04 |
| Claude Opus 4.6 | —† | —† | —† | —† |
| DeepSeek V3.2 | 53.00 | 17,030.96 | 6.00 | 19,209.09 |
| MiniMax M2.7 | 67.00 | 25,364.04 | 2.00 | 47,368.67 |